

Serie 8

Aufgabe 1:

Schreiben Sie eine Funktion `flat`, die eine beliebige verschachtelte Liste vollständig „glätten“ soll, das heißt, die Funktion soll eine Liste zurückgeben, die nur die tatsächlichen Elemente, aber keine weiteren Listen enthält.

Bsp.: Aus `["a", [1,2, [4,5], [0.5, "b"]], "c"]` soll `["a", 1, 2, 4, 5, 0.5, "b", "c"]` werden.

Gehen Sie dabei rekursiv vor und verwenden Sie die Funktion `isinstance(item, list)`. (Sie sollen dafür keine Bibliotheken verwenden!)

Aufgabe 2:

Die Recamán-Folge ist definiert durch:

- $a_0 := 0$.
- Wenn $n \geq 1$ und sowohl $a_{n-1} - n > 0$ als auch, dass $a_{n-1} - n$ nicht bereits vorgekommen ist (d.h., $\nexists k < n : a_k = a_{n-1} - n$), dann $a_n := a_{n-1} - n$.
- Ansonsten $a_n := a_{n-1} + n$.

Schreiben Sie eine rekursive und eine nicht-rekursive Funktion, die das n -te Folgenglied berechnet. Welche der beiden Implementierungen kann Folgenglieder für größere n berechnen, ohne einen Fehler zu erzeugen?

Aufgabe 3:

Mandelbrot: Schreiben Sie eine Funktion `mandelbrot(z, upperlim = 10)`, die eine komplexe Zahl z einliest und eine natürliche Zahl zurückgibt. Dabei soll folgendermaßen vorgegangen werden:

- Für festes $z \in \mathbb{C}$: Setze $a_0 := 0$ und rekursiv $a_{n+1} := (a_n)^2 + z$.
- Falls $|a_n| > 2$, soll n ausgegeben werden, ansonsten soll a_{n+1} berechnet werden.
- Falls nach `upperlim` vielen Iterationen die Grenze $|a_n| > 2$ noch nicht überschritten wurde, soll `upperlim` ausgegeben werden.

Sie dürfen weitere Hilfsfunktionen definieren und `mandelbrot` weitere Argumente übergeben. Verwenden Sie den folgenden Code, um Ihr Ergebnis zu visualisieren. Um ein schöneres Ergebnis zu erzielen, können Sie größere Werte von `upperlim`, `nx`, `ny` ausprobieren.

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
```

```

def mandelbrot(z: complex, upperlim: int = 10) -> int:
    # your code here

#####
#           Visualization           #
#####

# Set resolution of final image as (nx)x(ny) in pixels
nx = 100 # take nx many points in x-direction
ny = 100 # take ny many points in y-direction for  $z = x + iy$ 

x = np.linspace(-2, 0.5, nx)
y = np.linspace(-1, 1, ny)

F = np.empty((ny, nx), dtype=float)

for i, yy in enumerate(y):
    for k, xx in enumerate(x):
        z = xx + 1j * yy
        F[i, k] = mandelbrot(z)

X, Y = np.meshgrid(x, y)

plt.pcolormesh(X, Y, F)
plt.show()

```

Aufgabe 4:

Stern–Brocot: Schreiben Sie eine rekursive Funktion `stern(n)`, die das n -te Folgenglied der Stern–Brocot-Folge berechnet. Diese ist folgendermaßen definiert:

- $a_0 := 0, a_1 := 1$.
- Falls $n \geq 2$, unterscheiden wir:
 - Wenn $n = 2k$ gerade ist, dann ist $a_n := a_k$,
 - wenn $n = 2k + 1$ ungerade ist, dann ist $a_n := a_k + a_{k+1}$.

Verwenden Sie den folgenden Code, um Ihr Ergebnis zu visualisieren. *Anmerkung:* Die Quotienten a_n/a_{n-1} durchlaufen alle positiven rationalen Zahlen genau einmal, insbesondere sind a_n und a_{n-1} stets teilerfremd.

In []: `import matplotlib.pyplot as plt`

```

def stern(n: int) -> int:
    # your code here

#####
#           Visualization           #
#####

```

```

N = 1024 # upper limit, compute sequence until N

# Plot values a_n over index n

xs = list(range(N + 1))
ys = [stern(n) for n in xs]

plt.figure(figsize=(10, 4))
plt.scatter(xs, ys, s=5)
plt.xlabel("n")
plt.ylabel("a_n")
plt.show()

# Plot values a_n / a_{n-1} over index n

ratios = [stern(n) / stern(n-1) for n in range(2, N + 1)]
indices = list(range(2, N + 1))

plt.figure(figsize=(12, 4))
plt.scatter(indices, ratios, s=5)
plt.xlabel("n")
plt.ylabel("a_n / a_{n-1}")
plt.show()

```

Aufgabe 5:

Cantor-Menge: Schreiben Sie eine Funktion `cantor(interval, depth)`, die ein Intervall $[a, b] \subseteq \mathbb{R}$ als Tupel (a, b) erhält sowie eine natürliche Zahl `depth` als Iterationstiefe. Die Funktion soll eine Liste von Intervallen rekursiv zurückgeben.

- Für ein Intervall $[a, b]$ sollen die zwei Intervalle $[a, \frac{2}{3} \cdot a + \frac{1}{3} \cdot b]$ und $[\frac{1}{3} \cdot a + \frac{2}{3} \cdot b, b]$ hergenommen werden und darauf jeweils die Funktion `cantor` mit `depth = depth - 1` angewandt werden.
- Solange `depth > 1`, soll das obige iteriert werden. Wenn `depth = 1`, dann sollen nur die beiden Teilintervalle zurückgegeben werden.

Verwenden Sie den folgenden Code, um Ihr Ergebnis zu visualisieren.

```

In [ ]: import matplotlib.pyplot as plt

def cantor(interval, depth):
    # your code here

#####
#           Visualization           #
#####

interval = (0, 1)
max_depth = 4

plt.plot([interval[0], interval[1]], [max_depth, max_depth], linewidth=3,

```

```

for depth in range(1, max_depth+1):
    intervals = cantor(interval, depth)
    y = max_depth - depth
    for a, b in intervals:
        plt.plot([a, b], [y, y], linewidth=3, color='k')

labels = [""] + [f"depth {d}" for d in range(1, max_depth+1)]
plt.yticks(range(max_depth), labels[:0:-1])

plt.ylim(-1, max_depth+1)
plt.show()

```

Aufgabe 6:

Die Determinante $\det(A)$ einer quadratischen Matrix A kann folgendermaßen rekursiv berechnet werden:

- Für eine 1×1 -Matrix $A = [a_{11}]$ ist $\det(A) := a_{11}$.
- Für eine $n \times n$ -Matrix A mit $n > 1$ ist

$$\det(A) := \sum_{j=1}^n (-1)^{j+1} a_{1j} \det(M_{1j}),$$

wobei M_{1j} jene $(n-1) \times (n-1)$ -Matrix ist, die entsteht, wenn man die erste Zeile und j -te Spalte von A entfernt.

Schreiben Sie eine Funktion, die die Determinante auf diese rekursive Art berechnet.

Anmerkung: Dies ist der Laplace'sche Entwicklungssatz nach der ersten Zeile.